

Unit- 1 Introduction to Software Engineering

System or Software Development Life Cycle (SDLC)

SDLC is a systematic approach to create a structure for developer to develop high quality Software according to customer requirements.

The purpose of SDLC process is helps to produce quality product. Provides a guidance, policies and methodology. The number of phases in the system development may vary from model to model.

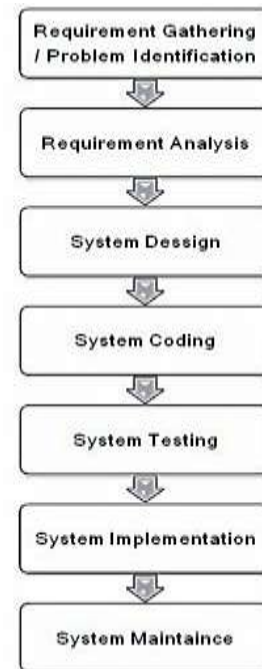
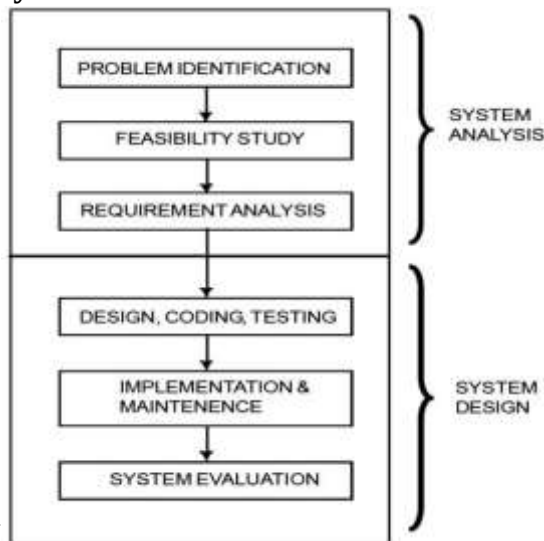
Each phase gives an output that becomes input of next phase.

➤ System Analysis

- 1) Problem Identification
- 2) Feasibility study and Cost Benefit Analysis
- 3) System requirement analysis

➤ System Design

- 4) System design specification and programming
- 5) System testing ,implementation & follow-up
- 6) System maintenance &Evaluation of the system.



System Analysis: System analysis means identification & study of the system for the purpose of achieving the goals.

It has modify interrelationships of components, deleting or merging of components.

It may upgrading of system.

Requirement gathering and analysis OR Problem Identification

To collect the requirements like input, output data and use of system through arrange meetings with managers, stakeholders and users.

Problem identification is the process of identifying the problem of existing system and determining why a new system is needed

Problem identification the focus is on:

- 1) Source of the problem: starting place of the problem may be internal or external.
- 2) Types of the problem: problems in the system are related to software Quality.
- 3) Nature of problem: find out the exact nature of the problem such as, how complex the problem.

Problem Identification helps:

1. It helps to setting proper system goals.
2. Determines boundary and scope of the system.
3. Checks limitations of available resources.

➤ Feasibility study and Cost Benefit Analysis

It defines developing the complete new system. The purpose of feasibility study is to determine the requested system successfully possible and evaluate the cost & benefits of the system.

Detailed feasibility study conduct by Steering Committee. Committee have system analyst, representatives from departments, and client.

Major aspects of feasibility study:

- **Economical:** Can we complete the project within the budget or not?
- **Operational:** Can we create operations which are expected by the client?
- **Technical:** Need to check whether the existing computer system can support the S/W.
- **Schedule:** Decide that the project can be completed within the given schedule or not.

System Requirement Analysis:

. System Analyst defines all the requirements of system known as **Software Requirements**

Specifications. Many tools are used to assess the user requirements such as Interview, Questionnaires, Developing various flow charts and Observation etc. This will make the system more users friendly.

Requirement analysis will determine

- 1 . What outputs are needed?
- 2 . What inputs are needed to obtain these outputs?
- 3 . Which operations it must perform to obtain O/P?
- 4 . What resource must be used?

System Design

System0 Design defines „How it is to be done“. System design is the most challenging and creative phase of SDLC. Transform the SRS document into logical structure, which contains detailed and complete specifications that can be implemented in a programming language.

Following tools are used for System design Flowchart, Data flow diagram (DFD), Data dictionary, Structured English, IPO (Input Processing and Output) and HIPO (Hierarchy of IPO) charts, Decision table and Decision tree.

Coding: Coding is translate system design into code using the programming languages. Tasks are divided into units or modules and assigned to the various developers. It is the longest phase of the SDLC process. Developer follows certain predefined coding guidelines, layout of programs, naming conventions for variables, classes and functions.

Testing: Testing is useful to detect errors in the software. The goal of testing is find coding errors and uncovered requirements in the S/W. Testing is a very critical and time-consuming activity. During the testing, actual result compared with the expected output. The final output of the testing phase is test report and error report. During testing phase importance-

- Testing should be planned before start.
- All tests trace customer requirements are fulfilled or not
- Code quality
- Testing are done by different types like functional, Non-functional, performance, system, and integration.

Implementation:

In implementation, system is installed at the user's place and implemented. It involves programmers, users and operations management.

It consists of installing hardware, programs, collecting data and run the system. Users are trained for operating and maintaining the system. Implementation includes

- Site preparation and Planning of implementation
- Purchase Hardware and software.
- Training to end users who will operate the new system.
- Users training, meetings & seminars
- Conversion of data files from old system to new system

Maintenance: Maintenance is necessary to remove errors and problems during working system and to adjust system in any working environment. The development team must maintain the software by fixing bugs and adding new features. It takes review of the system from time to time.

During the maintenance phase:

- Bug fixing - bugs are reported because of some development are not tested.
- Upgrade - Upgrading the application to the newer versions of the Software
- Enhancement - Adding some new features into the existing software

Types of maintenance:

1. **Corrective maintenance:** bugs (errors) are corrected
2. **Adaptive Maintenance:** certain modifications in the system.
3. **Perfective maintenance:** Enhancing performance and modifying the software to improve some of its qualities.
4. **Preventive Maintenance:** prevention of failures of equipment before it actually occurs.

Waterfall Model:

Waterfall Model is a **linear-sequential life cycle model or Classical model** that divides software development process into different phases. It was introduced in 1970 by Winston Royce. This model is basically used for small project and they have fixed requirements. One phase can be started after completion of the previous phase. That is the output of one phase will be the input to the next phase. Waterfall model is also useful to measure the progress of the project.

1. **Preliminary investigation:** All the detailed requirements are collected from the customer who wants to develop the software. To find out the source of the problem of the system.
2. **Requirement analysis:** system analyst analyses the requirements that whether requirements can be fulfilled or not. If customer requirements can be fulfilled within budget or not.
3. **System design:** Requirements translate into design. System architect designs software using different diagrams such as I/O Design, Procedures, class diagram, data flow diagram, state transition diagram etc. When a team is completely satisfied with the proposed design, then move to the next phase.
4. **System coding:** software design is translated into source code using any suitable programming language. Thus each designed module should be coded.

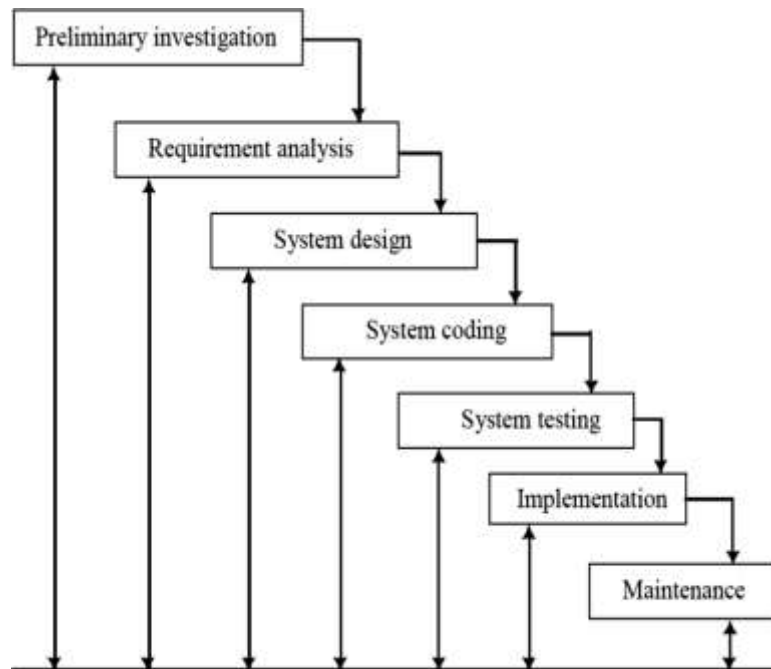


Fig : Waterfall Model

5. System testing: we test each unit or component and make sure the developed components are working as per customer requirements. Testing is identify and remove errors from the program. All the types of testing are done in this phase.

6. Implementation: system is actually installed at customers end and provides support. A feedback is taken from the customer to ensure the quality of the system and implementation in System.

7. Maintenance: The maintenance is a phase allows modifying the code for improve the quality of product. The modifications occur due to change requirements or adding new functionality started by the customer.

► **Advantages of waterfall model:**

- Easy to understand, manage and use.
- Systematic & sequential model
- Each phase has generates specific output.
- Phases are processed one at a time.
- Cost effective.
- Phases do not overlap.
- Used for smaller projects where requirements are Fixed & clear.

► **Disadvantages of waterfall model:**

- Not useful for large and ongoing project.
- Less effective if requirements are not clear at the beginning.
- Very difficult to move back to change on the previous phase.
- High risky.
- Less flexible.
- Customer interaction is less because Customer feedback only at the end of product.

► **When to use the waterfall model:**

- Use only when project is small and requirements are very clear and fixed.
- Product definition is stable.
- Technology is understood.
- For low budget projects.

○ **V-Process Model (Verification and Validation Model)**

Major disadvantage of waterfall model was testing done at the end of the development life cycle. It became very challenging & costly to fix the defects after the development process.

To overcome this problem, a new development model was introduced in 1980 by Paul Rook

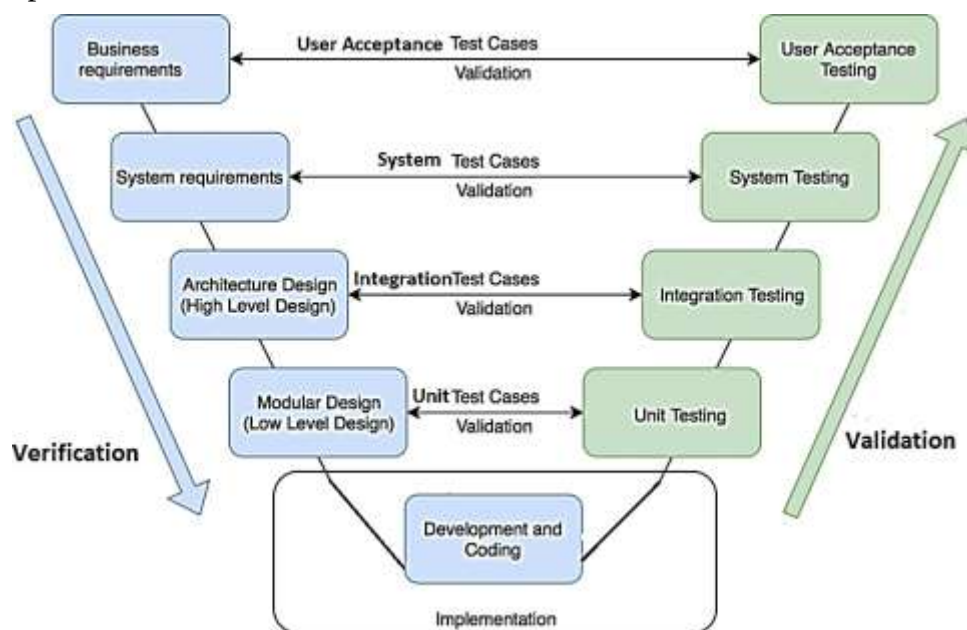
Called “V Model”. V-Model is an extension of the classic waterfall model **because Testing of the system is planned in parallel with a development phase** this model is processed step-by-step in which the next phase starts only after the current phase Verified and complete. Each phase of the development life-cycle has prepare a parallel test plan for the testing of system. Left hand side shows development activities and right hand side shows testing activities. Test levels shows by lines between both activities.

- **Verification:** Verification is a static analysis technique where testing is done without executing code. Examples- Reviews, Inspection and walkthrough.
- **Validation:** Validation is a dynamic analysis technique where testing is done by executing code. Examples-functional and non-functional testing techniques.

Development and Testing activities are done simultaneously.

1. Requirements Gathering and Analysis:

- **BRS-** Business Analysts will collect complete detailed requirement from the client and analyze. BA creates (BRS) Business requirement specification. Both test and development teams start their activities. Testing team will writes a user acceptance Test Plan for review BRS for finding any missing or wrong requirements.
- **SRS-** Manager and Analyst translate BRS(Business requirement specification) into SRS(Software Requirement Specification)
Using SRS, gathering the requirement which required for a system build. Testing team writes system test plan and test cases.



2. System Design: There are two types

- **Architectural Design** (High-level design): An architect's divide system design into components having different functionality. It consists of the list of modules, their interface relationships, dependencies, database tables, architecture diagrams, technology detail, etc.
- **Module Design** (Low-level design): Entire system splits into smaller modules. The detailed design of system components gets written down with greater clarity. It defines the actual logic for every component of the system.

3. Coding: Module design is converted into code by developers.

4. Implementation: Once coding is complete, the path of execution continues up the right side of the V where the test plans developed earlier to use.

Validation phases are on the right side of the V-shape. It consists of:

- **Unit testing:** testing at code level and removes the bugs at a beginning time.
- **Integration testing:** separate modules will be tested together to expose faults in the interfaces and in the interaction between integrated components.

- **System testing:** System testing will compare the system specifications against the actual system. It checks if the integrated product meets the specified requirements.
- **User Acceptance testing:** Ensures that the delivered product fulfills the client requirements.

► Advantages of V-model

- This saves a lot of time.
- Testing planned parallel to the development.
- Bugs detection in early phase.
- It provides better management for project risks
- Easy to Understand and manage.
- Works for small projects where requirements are fixed.

► Disadvantages of V-model

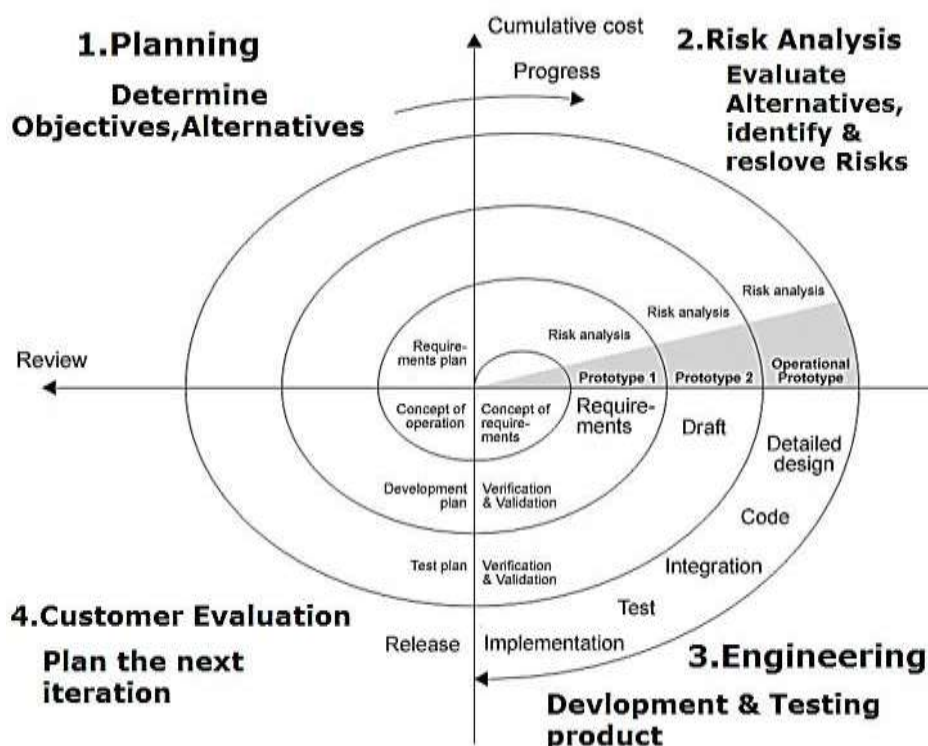
- less flexible.
- Not suitable for bigger and complex projects
- Expensive model as compared to waterfall model i.e. required lot of resources.
- V Model doesn't provide a clear path for problems found during testing phases.
- Adding or changing requirements at the middle of the process is difficult.

► When to use V-model

- Used for small or medium sized projects where requirements are clear and fixed.
- When sufficient technical resources are available with needed technical expertise.

○ Spiral model

The Drawback of traditional SDLC models is that they do not deal with the project risk. Software projects failed because project risks are neglected. When Barry Boehm tried to remove “project risk” factor into a spiral model presented in 1986. Spiral Model is a combination of a waterfall model and iterative model (1st ver. of SW design).



The spiral model is useful for identifying and removing high risk problems by carefully process design. . This model is used for large projects .

The phase is split into four sectors of major activities:

- 1. Planning** -Requirements are collected and analysis during the planning phase. Requirements like Business Requirement Specifications (BRS) and System Requirement specifications (SRS).It includes estimating the cost, schedule and resources for the iteration. It also involves understanding the system requirements for continuous communication between the system analyst and the customer.
- 2. Risk Analysis** - If any risk is found during the risk analysis then alternate solutions are suggested

and implemented. Risk Analysis includes identifying, estimating, monitoring technical and management risks.

3. Engineering: Mostly required features are developed and verified with the testing methodologies. It includes testing, coding and deploying software at the customer site.

4. Customer evaluation-Customer evaluates product and ensures that product meets all requirements of customer. If any changes required to the customer in the product, again all phases will be repeated. It is important to get feedback from the customer before releasing the product.

► **Advantages of Spiral Model**

- Fast and easy development
- Useful for large and mission-critical projects.
- Better risk analysis
- Flexible for frequently changes

► **Disadvantages of Spiral Model**

- Costly model to use.
- Time-consuming
- Risk analysis requires highly specific expertise.
- Project's success is highly dependent on the risk analysis phase.
- Expensive for small projects
- Proper customer communication required.

► **When to use Spiral model:**

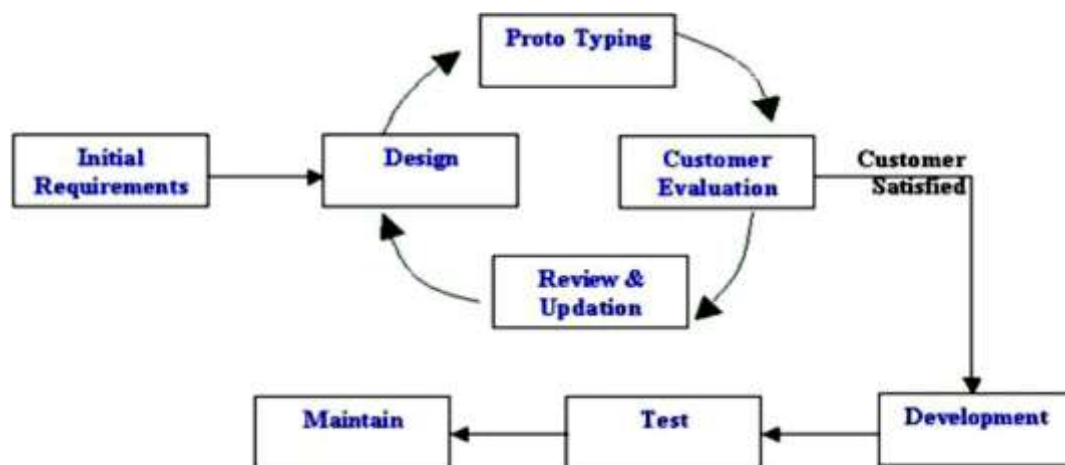
- Used for medium and large projects only.
- Used for high risk projects.
- When requirements are unclear and complex

○ **Prototype Model:**

The Prototyping Model (Working Model) is used when requirements are unclear from client at the beginning of the development of the system. When a prototype is developed using known unclear requirements and given to customer he uses the prototype and gets the feel of the software and then define clear requirements. Prototype gives just the idea about the final system.

Developers prepare design then build the prototypes. Prototypes are sent to Customer. Customer evaluates Prototypes and gives feedback. This model tries to increase the flexibility in the development process by allowing client to work together with developer and minimizes Risk factor.

1. Initial Requirement gathering- The prototyping model begins with initial requirement gathering. Developer and user meet and define the overall objective for the software, identify whatever requirements are known, and outline areas. Identify the requirements for making the system more feasible.



Proto Type Model

2. Design- The rough design of system is drawn for discussion. Developer and user jointly identify the data that are needed in the system. It specifies output that application must produce. Prototypes

are prepared to represent input screens format and output formats.

3. **Customer evaluation of prototype**- The prototype is evaluated by the user. User suggest some modify requirements in the software to be developed. User evaluate prototype and its features, operations. The developer better understand what are the needs of user.
4. **Review prototype** - The prototype is reviewed after getting information from user to check their likes and dislikes. Developer should understand the need of the customer properly for making modifications in the prototype.
5. **Implementation**- The process is repeated till both the user and developer find that all the necessary features are fulfilled or not repeating the steps if requirement are not fulfilled. Once both are sure then only prototype becomes ready for implementation.

► **Advantages of Prototype model**

- Strong communication between development and user.
- Prototyping makes requirements more clear.
- Time Saving.
- Used for large system
- Flexible design and development

► **Disadvantages of Prototype model**

- Client may be unknowledgeable.
- Changing requirements Increase the complexity in the system .
- Developer misunderstanding user objectives.
- Users may be confused because different prototypes.

► **When to use Prototype model:**

- The requirements are difficult to specify in advance.
- If there is possibility of changing the requirements significantly during the development and provide a feedback.
- A totally new system is proposed and quick development.
- When the desired system needs a lot of interaction with the users.

○ **Incremental model**

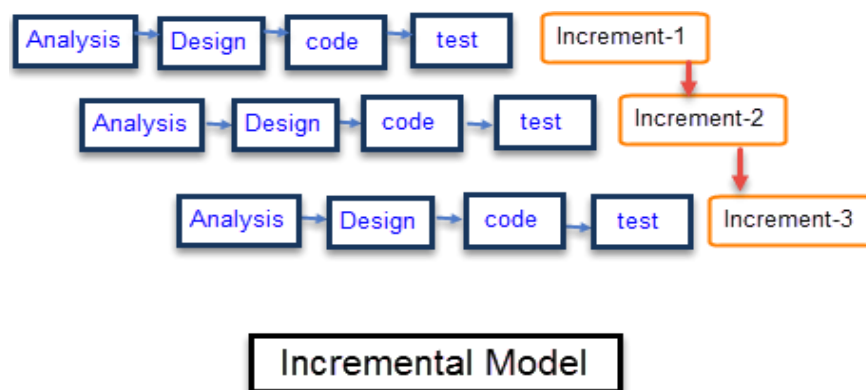
Software is divided into separate modules (components). Each module developed separate set of waterfall activities including requirements gathering and analysis, design, coding, implementation and maintenance.

When any component is ready, then delivered to the customer. Remaining components delivered to the customer one by one and integrating new components with old once.

Each module passes through the **requirements, design, coding and testing phases**.

Increment continuously release and adds its function to the previous released increment until all functionality has been implemented.

This model is useful when we have complete requirements as well as resources. Mostly used for



small project and if customers required some output after every phase.

Incremental Phases	Activities performed in incremental phases
Requirement Analysis	Requirement and specification of the software are collected

Design	Some high-end functions are designed during this stage
Code	Coding of software is done during this stage
Test	it goes through the testing phase

Example of incremental model:

Suppose we want to develop a web-based social network with the following functionalities-

- The user should signup the system.
- The user should log into the system
- can send or accept the friend request.

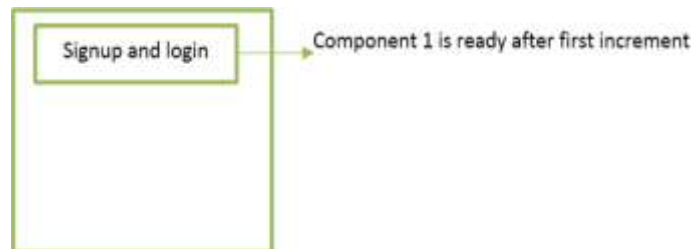
How can use incremental model in this development. ?

Solution: We need to convert this system into separate components- **Component 1:** Sign up and log in

Component 2: Send Friend request

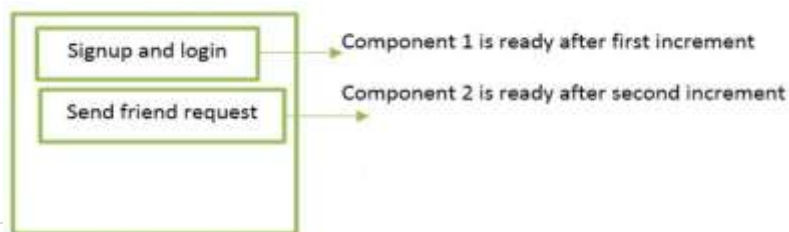
Component 3: Accept friend request

Start with component 1(signup and login). This component developed through the phases of requirements gathering, analysis, design, Code, test, implementation& maintenance. When this component is ready, we deliver this one component to the customer.

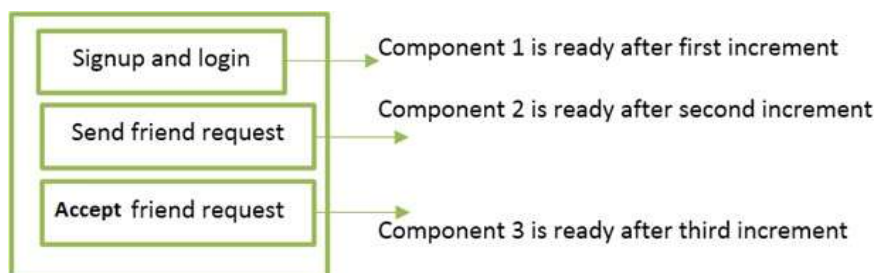


After that, develop another component 2 that sends the friend request. This component developed through the same phases of waterfall model. When this component is ready, we deliver this component to the customer.

After that, develop another component 3 that accepts a friend request.



This component developed through the same phases of waterfall model. When this component is ready, we deliver this component to the customer.



► Advantages of an incremental model

- Customer feedback is received after the delivery of each component.
- Risk of requirement changes is reduced
- More flexible
- Easy to test and debug
- Give quick results

► **Disadvantages of an incremental model**

- Requires good planning and design.
- Well defined module interfaces are required.
- More expensive as compared to waterfall model.
- This model is not suitable for small projects.

► **When to use incremental model?**

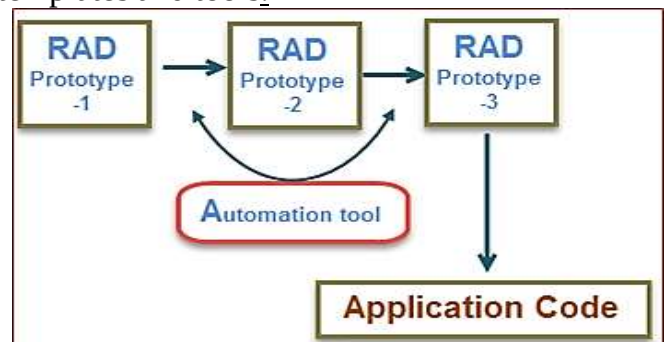
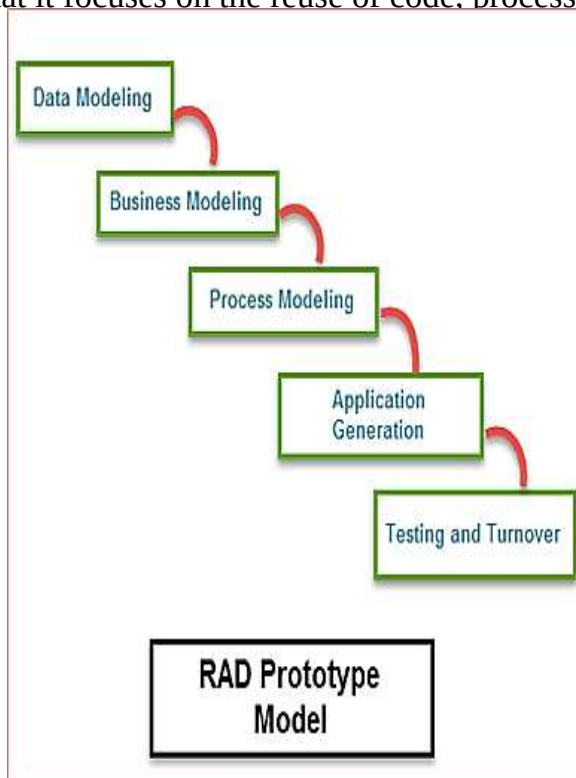
- When a customer has no problem with the budget but he demands more and more quality in software.
- Requirements of the complete system are clearly defined and understood.
- When software engineering team are not very well skilled or trained

○ **RAD (Rapid Application Development) Model**

RAD Model is similar to incremental model and prototyping. RAD is introduced by James Martin in 1991. In RAD Model, development should be done in specified time frame.

RAD Model is suitable for the small projects which have complete or fixed requirement. If the project is large, it can be broken down into small modules.

Each module can be assigned independently to separate teams. Thus, with a series of modules finally combined to the final project and delivered quickly. The major characteristic of the RAD model is that it focuses on the reuse of code, processes, templates and tools.



Phases of RAD Model:

1. **Data Modeling:** The information in the Data modeling phase is given into **the set of objects** and it is essential for the business. The attributes of each object are identified and defined the relationship between objects.
2. **Business Modeling:** Business modeling consists of the flow of information between various functions in the project. What type of information is produced by every function and which are the functions to handle that information.
3. **Process Modeling:** The data objects defined in the data modeling phase are changed to fulfill the information flow to implement the business model. The process description is created for adding, modifying, deleting or retrieving a data object.
4. **Application Generation:** The actual system is built and coding is done by using automation tools to convert process and data models into actual prototypes.
5. **Testing and turnover:** The prototypes are independently tested after each iteration so that the overall testing time is reduced. **The data flow and the interfaces between all the components are fully tested.** Hence, It reduces the risk of any major issues.

► **When to use RAD?**

- Reasonably well-known requirements
- When high skilled resources are available.
- High performance not required
- Low technical risks

► **Advantages of RAD**

- Fast application development and delivery.
- Less resource required
- Low cost and Good for small projects. It reduces development time as all prototypes are developed in parallel.
- Very flexible if any changes required.

► **Disadvantages of RAD**

- Required highly skilled developers and designers
- On each development phase client's feedback required.
- Using Automation tools project cost may be increased
- Automated code generation is very costly.
- Difficult to manage.

Agile:

The meaning of Agile is swift or versatile. "**Agile process model**" refers to a software development based on iterative development. Agile methods break tasks into smaller parts, It does not involve long term planning.

Each iteration is considered as a short time "frame" in the agile process model. The division of the entire project into smaller parts it helps to minimize the project risk.

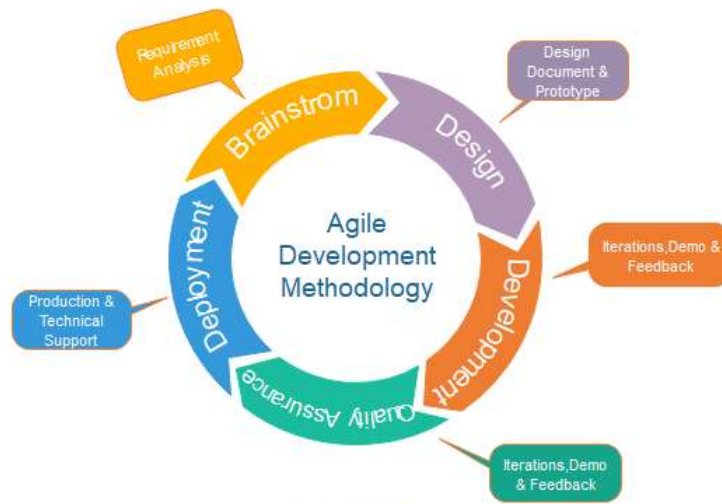


Fig. Agile Model

Following are the phases in the agile model are as follows:

1. Requirements gathering
2. Design the requirements
3. Construction/ iteration
4. Testing/ Quality assurance
5. Deployment
6. Feedback

1. Requirements gathering: In this phase, you must define the requirements. You should explain business opportunities and plan the time and effort needed to build the project.

2. Design the requirements: When you have identified the requirements. You can use the user flow diagram or the high-level diagram to show the work of new features and show how it will apply to your existing system.

3. Construction/ iteration: When the team defines the requirements, the work begins. Designers and developers start working on their project. The product will undergo various stages of improvement, so it includes simple, minimal functionality.

4. Testing: In this phase, the Quality Assurance team examines the product's performance and looks for the bug.

5. Deployment: In this phase, the team issues a product for the user's work environment.

6. Feedback: After releasing the product, the last step is feedback. In this, the team receives feedback about the product and works through the feedback.

🔍 Software Requirements:

Software requirement is a process of defining expectations of the users from the software system. After collected all the requirements, system to be developed.

These system requirements are classified in functional requirements and non-functional requirements.

▪ **Functional Requirements** - Functional requirements defines the functions. It has to be performed, the software product. How system worked on accepting inputs from the user, and produces the desired output.

- **Example:** Hospital management system.
- Search option given to doctor to retrieve information of patients.
- Doctor should be able to Mail the report to other doctors.

• **Non-functional Requirements** – Nonfunctional requirements are essential to performance of software product under different types of environmental conditions are called as nonfunctional requirements.

• **Example-** maintainability, portability, usability, reliability

• **User Requirements:** It defines expectation of customer.

• What type of input and output requirements of the user?

• It also includes source, formats, contents, etc.

◦ User requirements are defined: 1. Attractive 2. Simple to use 3. clear to understand

System Requirement: System Requirement is a specific and detailed description of what a software application or a technical system needs to do. These requirements provide the functionalities behaviors and technical specification. They help the software can functions within the desired technical environment.

Example- Playing a game on computer system or phone it has requirement of at least 4 GB RAM and Graphics card

🔍 Fact finding techniques

It is very difficult to collect information from large and complex organizations. So analyst make a clear plan to collect information.

Fact finding techniques means it is **A specific method used for finding information of the system are called fact finding techniques.** The analyst can be use multiple techniques for investigation.

Fact gathering techniques are:

1. Interview
2. Questionnaire
3. Record review
4. Observation



1. Interview-

In interview technique, analyst collects information about the current working systems form **face-to-face interviews with persons or groups of users**. The purpose of interview technique is to find and verify facts, identify requirements, gather ideas, suggestions and existing problems etc. In this technique good communication skills required for interaction between analyst and user. The analyst is also observe body language of interviewee (Users).

Types of interview –

Structured Interview	Unstructured Interview
Fixed type of questions are asked	Different type of questions are asked
Simple to manage	Not simple to manage
Specific information is collected	Possibility of unnecessary information
Interview period is short	No limit on interview period
Lot of study & preparation is required	No such preparation is required

► **Advantages -**

- More accurate and reliable information is collected.
- Interviewer modify questions during interview.
- Interviewer observe interviewee's body language.

► **Disadvantages -**

- Time consuming and costly
- Requires highly skilled interviewer

2. Questionnaire -Questionnaire technique is a structured and formal method of collecting data. It is not possible to conduct interviews of all the users. In such cases, system analyst use questionnaire. This method taking information from the person in written format.

There are two types of questions-

• **Open ended questions (free-format):** Questions are given without any choice of answers. User has to write the answers in own words on the given space.

• **Closed ended questions (fixed-format):** Questions are given with choice of answers for each question and the user has just to mark the correct response. Closed ended questions have fixed responses such as following types:

1. **Fill In the blanks questions-** gives specific information. Responses can be statistically analyzed.
2. **Yes/No type questions-** Answered by yes or no, otherwise I don't know.
3. **Multiple choice questions-** Select specific options for an answers. Advantage is faster tabulation and less analyst bias.
4. **Rating scales questions** response is given a range of 1-5.

► **Advantages of questionnaires**

- Large amounts of information can be collected in a short period of time.
- Less expensive than interview technique.
- Time saving as compared to other techniques.

► **Disadvantages of questionnaires**

- Asking only limited amount of questions without explanation.
- Difficult to design questionnaire format.

3. Record Review – In this type the Information related to system is already available in some type of documents, records and files (like system user manual, system review/audit, brochures etc.). That information and documents examined by analyst. That documents provides a valuable information about the system.

4. Observation -Observations are one of the most effective tools for analyst where an analyst personally goes to the site and observe the functions of the system.

This is a highly successful technique, because developers have the chance of experiencing the working system.

Advantages

- Observer can see exactly what is being done
- Observer can also obtain data describing the physical environment of the task
- Observer can do work measurement

Disadvantages

- Operations run less smoothly because people might be nervous during observation.
- May wrongly observing certain tasks.
- Time consuming

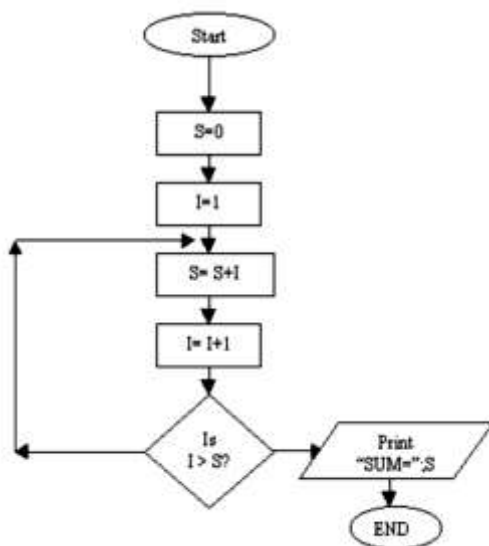
Analysis and Design Tools:

System Analysis and Design is used various tools for representing complex processes and procedures that involved in system. Using some analysis & design tools i.e. Flowcharts, Decision Tables, Decision Trees and Structure charting Techniques.

Flowcharts

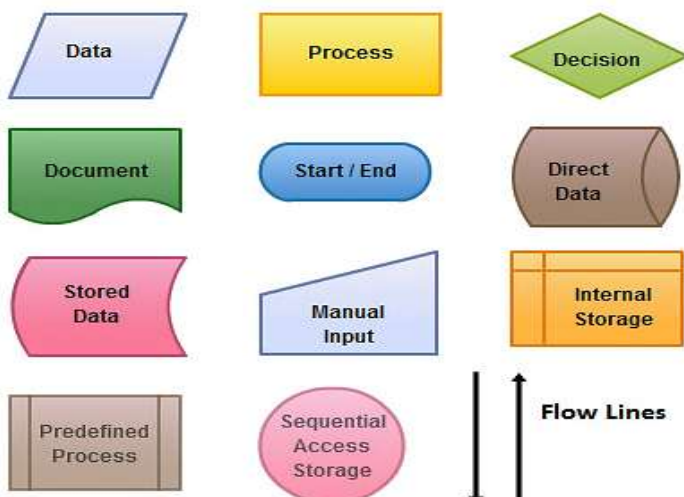
A flowchart consists a set of 'symbols' which is connected by arrows. In flowchart, boxes are used to represent different operations and the arrows are used to represent the sequence of these operations. The purpose of flowchart is graphically represent logical flow of data in the system.

1. Draw Flowchart of Sum of two nos.



2. Draw flowchart of Average of two nos.

Symbols used in a flow charts:



General guidelines to prepare Flowcharts:

1. Listing all requirements in proper logical order in detail.
2. Use suitable standard flowcharting symbols and templates.
3. Avoid cross lines
4. Use connectors to reduce the number of flow lines.

► Advantages of flowcharts:

- Excellent way of communicating logic of a program.
- Problem can be analyzed in more effectively.
- Flowchart makes program or system maintenance easier.
- It helps the programmer to write the program code into any programming language.

► Disadvantages of Flowcharts:

- Drawing flowchart is a time-consuming task.
- Difficult to alter or redraw the flowchart.
- It requires special tools to draw the necessary symbols.

Decision tables: Decision Table represents conditions and actions in a tabular format that is Rows and Column.

Decision Table

❑ Components of Decision Table:

- ❑ **Condition stub:** It is the upper left quadrant which lists all the condition to be checked.
- ❑ **Action Stub:** It is the lower left quadrant which lists all the action to be carried out to meet such condition.
- ❑ **Condition Entry:** It is the upper right quadrant which provides answers to questions asked in condition stub quadrant.
- ❑ **Action Entry:** It is the lower right quadrant which provides the appropriate action resulting from the answers to the conditions in the condition entry quadrant.

Condition stub	Condition entry
Action stub	Action entry

Decision table is divided into two parts:

- Condition
 - Action
-
- **Condition:** In this type it shows and specifies all conditions that are applied to the input data. It is also divided into two parts-
 - **Condition Stub** – It shows the various possible conditions exist in given data.
 - **Condition Entry** - all possibilities of yes and no answers related to the condition stub.
 - **Action:** The action part is subdivided into two parts-
 - **Action Stub** - shows the various actions taken against different conditions.

- **Action Entry** - is used to find out which action is taken corresponding to a particular set of conditions. (X) or () ✓ Action Applicable, **Dash (-)**: action Not Applicable. Physical layout of decision table is as follows.

Creating Decision Table

- Identify all possible conditions.
- Determine actions for identified all conditions
- Create Maximum possible rules.

Determine the rules in a decision table by the formula:

Number of rules = 2^N Here N represents the number of condition.

A decision table with four conditions has 16 Rules. ($2^4 = 2 \times 2 \times 2 \times 2 = 16$) and rules with six conditions has 64 rules and eight conditions yield 256 rules.

Examples: For 2 conditions there are $2^2=4$ possible combinations.

Condition 1	Y	Y	N	N
Condition 2	Y	N	Y	N

CASE 1: Consider case of a bank which grants loan to the customer under the following conditions-

- 1) If a customer has an account with the bank and has no previous loan outstanding, the loan will be granted.
- 2) If a customer has an account with the bank but some amount is outstanding from previous loans then new proposal's loan will be granted if special management approval is there.
- 3) In all other situations loan applications are rejected.

Draw decision table:

Step- 1: List of conditions and Actions

Condition:	1. Customer has an account. 2. No previous loan outstanding 3. Customer has management approval.	Action:	1. Grant loans 2. Reject loans.
-------------------	--	----------------	------------------------------------

Step-2: Combine conditions which describe two possibilities of a single condition.

Rules	Condition	Action
1	Customer has account and no dues from pervious loan	Grant Loan Application
2	Customer has account and some dues from pervious loan. He has management approval	Grant Loan Application
3	Customer has account and some dues from pervious loan. He has no management approval	Reject Loan Application
4	Customer has no account	Reject Loan Application

Header	Decision Table	Rule 1	Rule 2	Rule 3	Rule 4
CS	Condition1: Customer has Bank A/C	Y	Y	Y	N
	Condition2: Customer has no dues	Y	N	N	-
	Condition3: Customer has mgt. approval	Y	Y	N	-
AS	Action 1: Grant Loan	X	X	-	-
	Action 2: Reject Loan Application	-	-	X	X

Y: Yes, N: No and X: Applicable, -: Not Applicable

Decision Table

Doctor's Clinic

Condition	Rule 1	Rule 2	Rule 3	Rule 4
Age≤10	Yes	-	-	-
Age>10 & Age≤40	-	-	Yes	Yes
Age>40	-	Yes	-	-
Regular	-	-	Yes	No
Action				
Fee	No	Yes	Yes	Yes
Discount	-	10%	5%	No

► Advantages of Decision Tables:

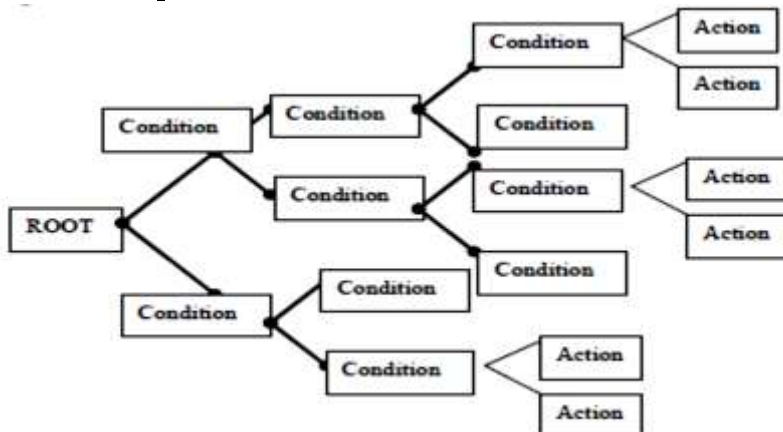
- Decision rules are clearly structured
- Documentation is easily prepared, changed or updated
- Easy to use
- Enable more compact documentation
- Easy to understand by non-computer users and managers

► Disadvantages of Decision Tables:

- Does not show the flow
- Cannot list all the alternatives
- Not easy to translate it

+ Decision Tree

Decision tree is a tree like structure that represents the various conditions and the subsequent possible actions.



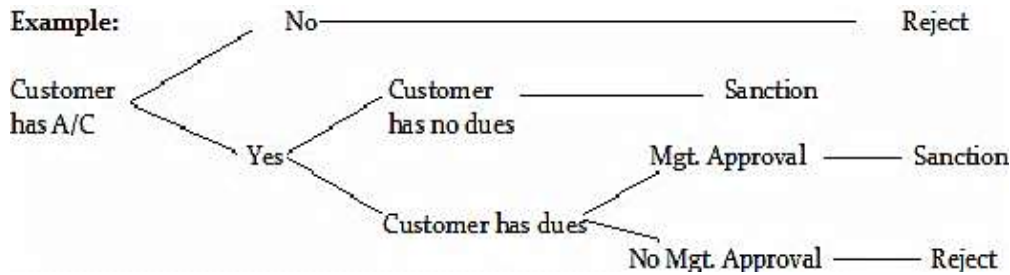
Decision Tree is tool for documenting procedures where actions needs to be taken Depending on a number of conditions. This method show the relationship of each condition & it's allowed actions.

The root is on the left side and the actions are shown at the right side Decision tree start from the root (starting point of the decision sequence) and proceed towards the various possible nodes and sub nodes. The size of the tree will depend upon the number of conditions and actions. Each condition is express in two ways, namely T/F or Y/N.

This decision tree representation form is very beneficial to the analyst for show all the given condition in a logical format

Example: Consider case of a bank which grants loan to the customer under the following conditions-1) If a customer has an account with the bank and has no previous loan outstanding, the loan will be granted.

- 2) If a customer has an account with the bank but some amount is outstanding from previous loans then new proposal's loan will be granted if special management approval is there.
- 3) In all other situations loan applications are rejected Draw Decision Tree and Table?



Draw decision table:

Step- 1: List of conditions and Actions

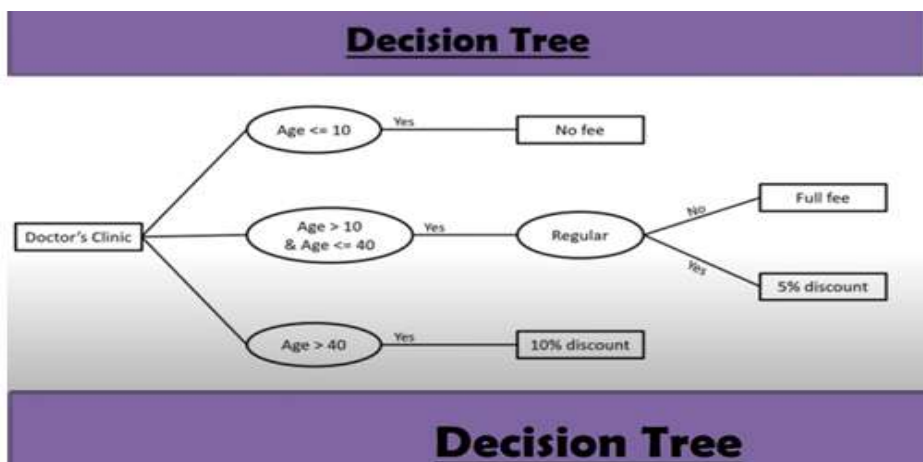
Condition:	1. Customer has an account. 2. No previous loan outstanding 3. Customer has management approval.	Action :	1. Grant loans 2. Reject loans.
-------------------	--	-----------------	------------------------------------

Step-2: Combine conditions which describe two possibilities of a single condition.

Rules	Condition	Action
1	Customer has account and no dues from pervious loan	Grant Loan Application
2	Customer has account and some dues from pervious loan. He has management approval	Grant Loan Application
3	Customer has account and some dues from pervious loan. He has no management approval	Reject Loan Application
4	Customer has no account	Reject Loan Application

Header	Decision Table	Rule 1	Rule 2	Rule 3	Rule 4
CS	Condition1: Customer has Bank A/C	Y/T	Y/T	Y/T	N/F
	Condition2: Customer has no dues	Y/T	N/F	N/F	-
	Condition3: Customer has mgt. approval	Y/T	Y/T	N/F	-
AS	Action 1: Grant Loan	X	X	-	-
	Action 2: Reject Loan Application	-	-	X	X

Y: Yes, N: No and X: Applicable, -: Not Applicable



Advantages of Decision Tree:

- It expresses the logic of 'if then else' in pictorial form, showing conditions & actions.

- Outcome of another decision.
- It helps the analyst to identify the actual decision to be made.
- It is used to verify problems that involve a few complex decision and limited actions.

► **Disadvantages of Decision Tree:**

- The lack of decision tree, absence of information in its format to take what other combinations of conditions to test.
- A large number of branches with many paths will confuse rather than help in analysis.

Structured English: Structured English is similar to programming language such as Pascal.

It does not have strict syntax programming rules.

It provides the precise description of process.

The structured English should be understandable to the user.

Example-

Conditional statement If -Then

BEGIN IF

IF Customer age > 65

THEN Billing Rate= Senior Citizen Rate

ELSE Billing Rate =Standard Rate

END IF

If customer pays advance

Then

Give 5 % Discount

Else

If purchase amount >=10,000

Then

If the customer is a regular customer,
then give 5% discount

else no discount

end if

else no discount

end if

end if

Structure Chart: Structure Chart are designing tools for Describing the system according to functions. It describes the software architecture.it is a next step of DFD.Structured chart consists of rectangular boxes that represents the modules connecting arrows and lines.

Symbols in structured chart:

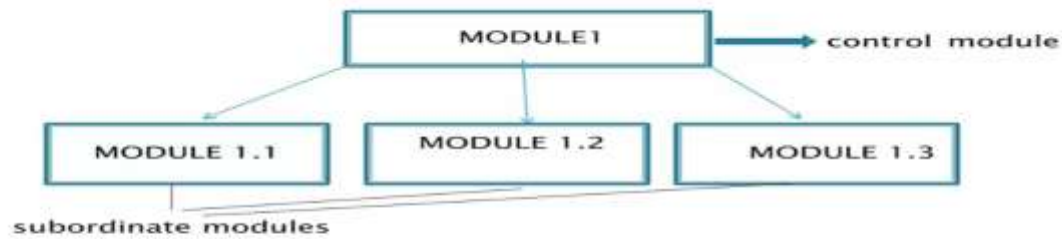
1) Module: It represents process or task of system

2) Loop: It represents repeated execution of a module. A curved arrow represents loop in the module.

3) Conditional call: It represent that the control module can selects any of its sub modules based on some condition.

4) Data Flow: It represents flow of data in the modules

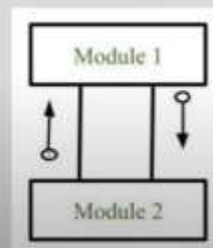
5) Control Flow: It represents flow of control in the module.



- **Control Module:** It is a higher-level module that directs lower-level modules.
- **subordinate modules:** These are the lower level modules.

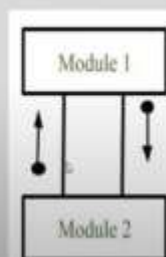
Data Flow

- It represents the flow of data between the modules.
- It is represented by directed arrow with empty circle at the end.



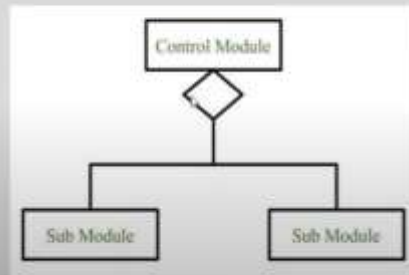
Control Flow

- It represents the flow of control between the modules.
- It is represented by directed arrow with filled circle at the end.



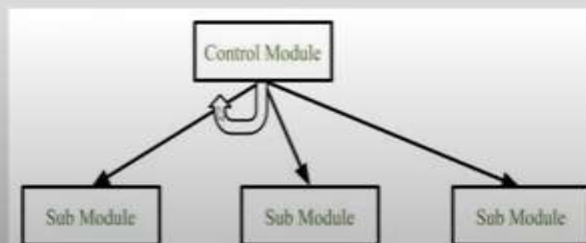
Conditional Call

It represents that control module can select any of the sub module on the basis of some condition.



Loop (Repetitive call of module)

- It represents the repetitive execution of module by the sub module.
- A curved arrow represents loop in the module.
- All the sub modules cover by the loop repeat execution of module.



Example

